

# MAP ASSIGNMENT-5

## 1) Configuring a Mule 4 Application for File Read and Write Operations (with Flow Diagram)

In Mule 4, we can easily perform file operations such as reading from and writing to files by designing a simple flow. Below is the step-by-step procedure to configure these operations.

### Step 1: Create a New Mule Project

- Open **Anypoint Studio** and create a new project named **fileoperations**.

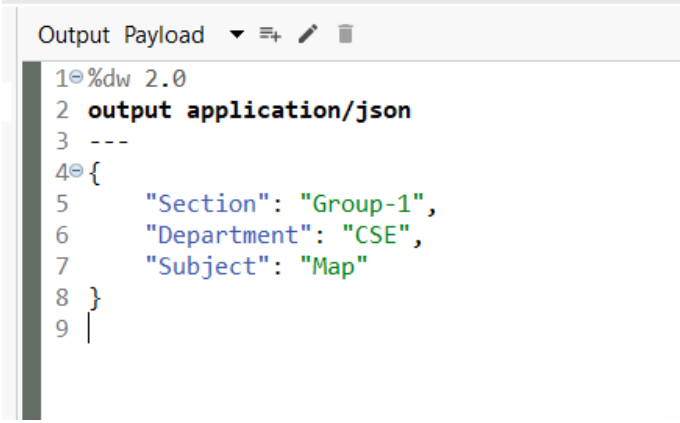
### Write Operation Flow

#### 1. Add an HTTP Listener

- Drag the **HTTP Listener** component onto the canvas.
- Set the **Path** to `/write`.
- This component will listen for incoming requests to trigger the write operation.

#### 2. Add a Transform Message Component

- Place a **Transform Message** component after the listener.
- Use DataWeave code here to generate a **JSON payload** containing student details.

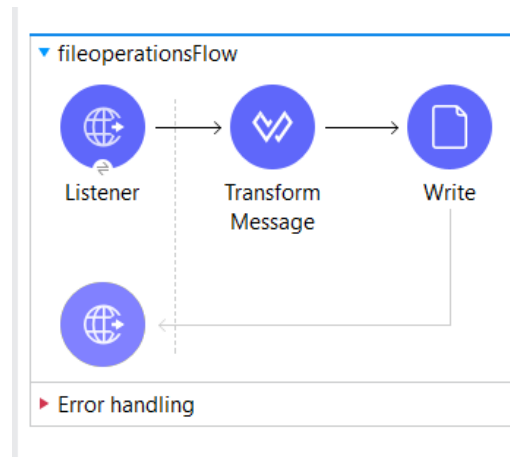


```
Output Payload ▼ ⚙ ✎ 🗑
1 %dw 2.0
2 output application/json
3 ---
4 {
5     "Section": "Group-1",
6     "Department": "CSE",
7     "Subject": "Map"
8 }
9 |
```

#### 3. Add a Write Component

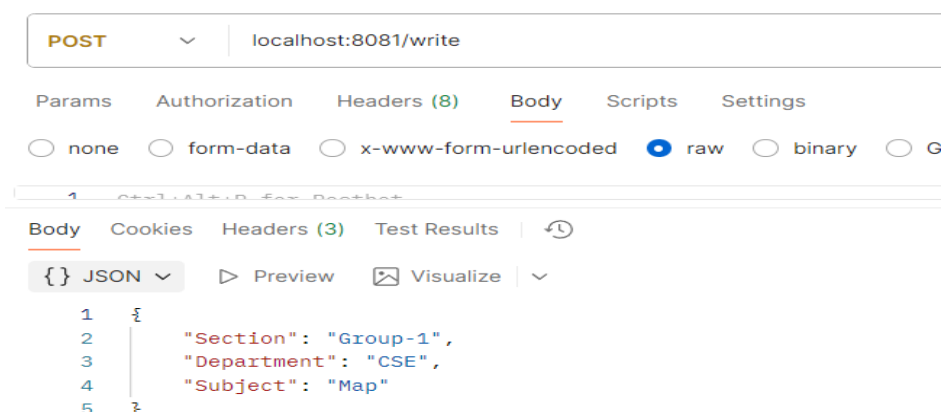
- Drag the **Write** component next in the flow.
- Configure the following:

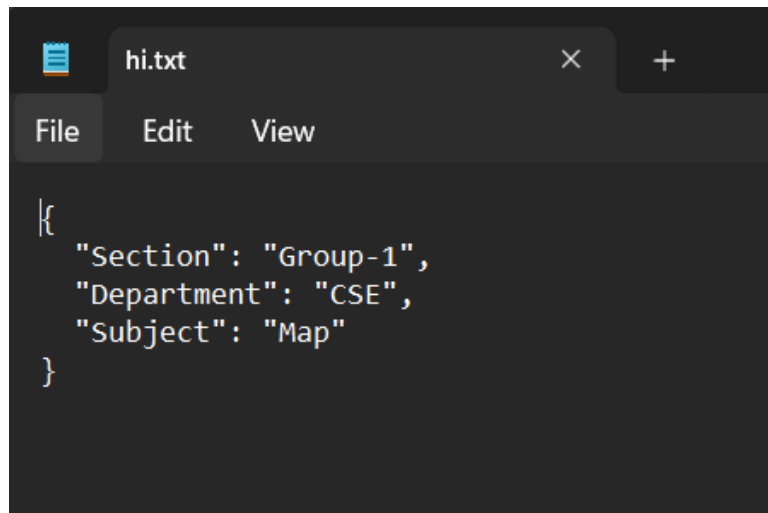
- **Path:** C:\AnypointStudio\FileOps\hi.txt
  - **Content:** payload
  - **Write Mode:** Overwrite (default setting)
- This step creates a file named hi.txt (or overwrites it if it exists) and writes the generated JSON data into it.



#### 4. Deploy and Test the Application

- Save and run the Mule app.
- Open **Postman** and send a request:
  - **Method:** POST (or GET, as per configuration)
  - **URL:** http://localhost:8081/write
- A file named hi.txt should now be created at the specified location with the JSON content inside.





```
{
  "Section": "Group-1",
  "Department": "CSE",
  "Subject": "Map"
}
```

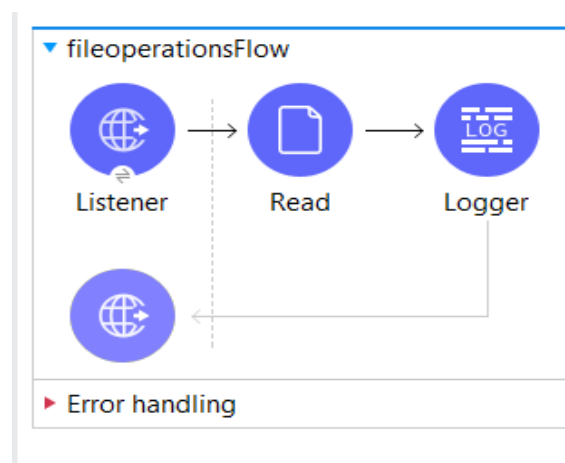
## Read Operation Flow

### 1. Add Another HTTP Listener

- Add a new **HTTP Listener** component.
- Configure its **Path** as /read and **Method** as GET.

### 2. Add a Read Component

- Drag the **Read** component after the listener.
- Set the **Path** to C:\AnypointStudio\FileOps\hi.txt.
- This reads the contents of the previously created file.

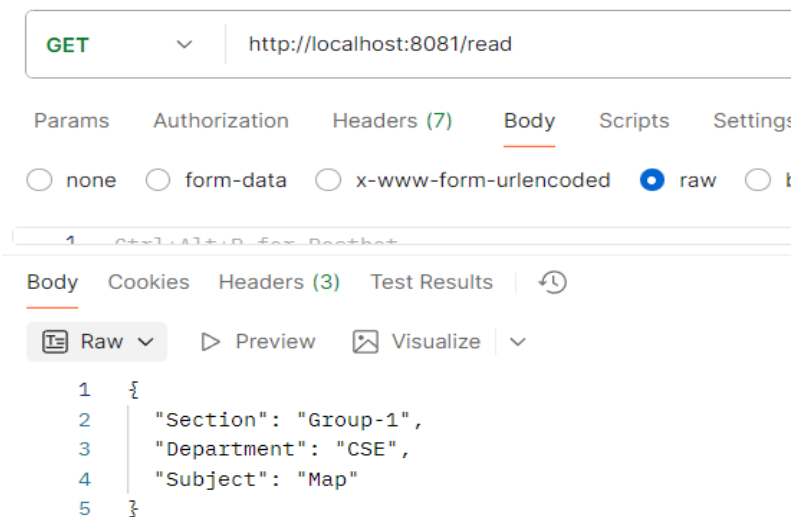


### 3. Add a Logger Component

- Drag a **Logger** component after the Read component.
- Set the **Message** as #[payload].
- The logger will print the contents of the file to the console.

#### 4. Test the Read Operation

- Run the Mule application and open **Postman**.
- Send a GET request to `http://localhost:8081/read`.
- The contents of `hi.txt` will be read and displayed in the console logs.



## 2) Updating Files Using Flow Triggers in Mule 4

In practical use cases, applications often need to **automatically detect and process new or modified files**—such as CSV, JSON, or text files—without manual intervention. MuleSoft offers the **“On New or Updated File”** component for this purpose, which continuously monitors a directory and triggers flows whenever changes occur.

### Use Cases

- Auto-processing student records as soon as new files arrive.
- Updating employee or payroll data when a file is modified.
- Real-time monitoring of logs or transaction files.

### Steps to Create the Flow

#### 1. Add “On New or Updated File” Component

- Drag and drop the **On New or Updated File** component into the flow.
- Configure it as follows:

- **Directory:** Specify the folder to be monitored.
  - **Frequency:** 5
  - **Start Delay:** 0
  - **Time Unit:** Seconds
- Mule will now check the folder every 5 seconds for any new or updated files.

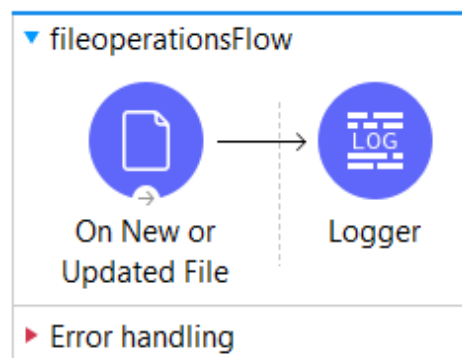
Scheduling Strategy Fixed Frequency

|              |         |
|--------------|---------|
| Frequency:   | 5       |
| Start delay: | 0       |
| Time unit:   | SECONDS |

Post processing action

## 2. Add a Logger Component

- Add a **Logger** component after the file trigger.
- Set the message to payload.
- This will log the contents of the detected file whenever a change occurs.



## 3. Run and Test the Flow

- Start the Mule application.
- Place a new file in the monitored directory.
- Check the logs – the file content will appear as the payload.

```
}
INFO 2025-10-02 11:30:59,643 [[MuleRuntime].uber.10: [fileoperations].fileoperationsFlow.CPU_LITE @631c5944] [processor: fileoperationsFlow/processors/0; event: 2b534;
  "Section": "Group-1",
  "Department": "CSE",
  "Subject": "Map"
}
INFO 2025-10-02 11:31:04,662 [[MuleRuntime].uber.10: [fileoperations].fileoperationsFlow.CPU_LITE @631c5944] [processor: fileoperationsFlow/processors/0; event: 2e50d;
  "Section": "Group-1",
  "Department": "CSE",
  "Subject": "Map"
}
```

#### 4. Update the File and Observe Changes

- Modify the contents of the existing file.
- The flow will trigger again, and the Logger will display the updated payload automatically.

```
fileoperations [main:operations] [prod:CS000]
INFO 2025-10-02 11:31:34,645 [[MuleRuntime].uber.10: [fileoperations].fileoperationsFlow.CPU_LITE @631c5944] [processor: fileoperationsFlow/processors/0; event: 40300:
  "Section": "Group-1",
  "Department": "CSE",
  "Subject": "Map"
}
INFO 2025-10-02 11:31:39,656 [[MuleRuntime].uber.10: [fileoperations].fileoperationsFlow.CPU_LITE @631c5944] [processor: fileoperationsFlow/processors/0; event: 432cc:
  "Section": "Group-1",
  "Department": "CSE",
  "Subject": "Map"
}
INFO 2025-10-02 11:31:44,647 [[MuleRuntime].uber.10: [fileoperations].fileoperationsFlow.CPU_LITE @631c5944] [processor: fileoperationsFlow/processors/0; event: 46267:
  "Section": "Section-1",
  "Department": "CSE",
  "Subject": "Map"
}
```

#### Benefits of Using “On New or Updated File”

- Enables **real-time file tracking** and automation.
- Removes the need for **manual checks or repeated read operations**.
- Can be paired with **DataWeave** to transform, validate, or enrich data before further use.
- Useful in batch jobs, reporting workflows, and system integrations.